

Credit Card Fraud Detection Model Deployment Manual

Team Members: Xavier Jackson, Tyler Helmrich, Arth Patel, Zenil Moradia, Jay Patel

Version: 1.0

Introduction

This manual provides a step-by-step guide for deploying the Credit Card Fraud Detection Application. It covers requirements, installation, configuration, and verification.

System Requirements

Hardware

- **Minimum:** 4 CPU cores, 8 GB RAM, 50 GB disk space
- **Recommended:** 8 CPU cores, 16 GB RAM, 100 GB disk space

Software

- **Operating System:** macOS 26 Tahoe (Sept 2025), iOS 26 (2025)
- **Python:** 3.10+
- **Database:** Supabase
- **Frontend:** macOS with Xcode 15+ (for SwiftUI build)

Dependencies

Python (typical):

- pandas, numpy, xgboost, joblib, requests, streamlit

- scikit-learn (train/test split, scaling, metrics)
- imbalanced-learn (SMOTE)

Xcode frameworks / packages:

- SwiftUI, Charts, Foundation, UserNotifications
- Pushy SDK

Supabase Edge Function:

- @supabase/supabase-js
- deno/std HTTP server

Installation Steps

Backend API

Clone repository

git clone <https://github.com/htmwf/F2025-Team1.git>

cd Code

Install dependencies

pip install -r requirements.txt

Frontend (iOS)

1. Open the project in Xcode.
2. Install Pushy SDK:
 - a. Xcode > File > Add Package Dependencies
 - b. Add: <https://github.com/pushy/pushy-sdk-ios>
3. Add Pushy to the app target:
 - a. Select the project in the file tree
 - b. Targets > CreditCardFraudApp > General
 - c. Frameworks, Libraries, and Embedded Content > +
 - d. Add Pushy

```
xcodebuild -scheme CreditCardFraudApp \  
-destination 'platform=iOS Simulator,name=iPhone 17'
```

Database Setup (Supabase)

1. Supabase Dashboard > create a new project.
2. Copy the required values into your environment/secrets:
 - a. SUPABASE_URL
 - b. SUPABASE_SERVICE_ROLE_KEY

Define Tables

CreditCards Table - Stores card metadata linked to users

```
create table public."CreditCards" (  
  "ccNumber" varchar not null,  
  exp varchar not null,  
  sec varchar not null,  
  iss varchar not null,  
  created_at timestamptz not null default now(),  
  "userId" uuid not null default gen_random_uuid(),  
  constraint CreditCards_pkey primary key ("ccNumber"),  
  constraint CreditCards_userId_fkey foreign key ("userId")  
    references "Users" (id)  
    on update cascade  
    on delete cascade  
) tablespace pg_default;
```

Transactions Table - Stores transaction records, fraud flags, and geolocation data

```
create table public."Transactions" (  
  "userId" uuid not null default gen_random_uuid(),  
  "ccNumber" varchar not null,  
  date date not null default now(),  
  time time with time zone not null default now(),
```

```
longitude double precision not null,  
latitude double precision not null,  
amount integer not null,  
created_at timestamptz not null default now(),  
fraud_flag boolean default false,  
fraud_reason text,  
fraud_checked_at timestamptz,  
txn_id uuid default gen_random_uuid(),  
ts timestamptz,  
"locationName" text not null default 'Whole Foods',  
city text not null default 'New York',  
state text not null default 'New York',  
constraint transactions_pkey primary key ("userId", "ccNumber", "time"),  
constraint transactions_ccnumber_fkey foreign key ("ccNumber")  
    references "CreditCards" ("ccNumber")  
    on update cascade  
    on delete cascade,  
constraint transactions_userid_fkey foreign key ("userId")  
    references "Users" (id)  
    on update cascade  
    on delete cascade  
) tablespace pg_default;
```

-- Indexes

```
create index if not exists idx_transactions_date_fraudflag  
on public."Transactions" using btree (date, fraud_flag);
```

```
create index if not exists idx_transactions_txn_id  
on public."Transactions" using btree (txn_id);
```

```
create index if not exists idx_transactions_created_at  
on public."Transactions" using btree (created_at);
```

```
create index if not exists idx_transactions_ts_fraud  
on public."Transactions" using btree (fraud_flag, ts);
```

-- Trigger

```
create trigger transactions_set_ts_before
```

before insert or update on "Transactions"
for each row execute function transactions_set_ts();

Users Table - Stores registered users and admin roles

```
create table public."Users" (  
  id uuid not null default gen_random_uuid(),  
  name varchar not null,  
  created_at timestamptz not null default now(),  
  "isAdmin" boolean not null default false,  
  constraint Users_pkey primary key (id)  
) tablespace pg_default;
```

Supabase Edge Function Setup

1. Supabase Dashboard > your project > **Edge Functions**.
2. Create a new Edge Function (example name: app-middleware).
3. Locate Supabase project credentials:
 - a. SUPABASE_URL: Supabase Dashboard > your project > **Settings** > **API** > Project URL
 - b. SUPABASE_SERVICE_ROLE_KEY: Supabase Dashboard > your project > **Settings** > **API** > Project API keys > **service_role** (secret)
4. Set the required Edge Function secrets/environment variables:
 - a. SUPABASE_URL
 - b. SUPABASE_SERVICE_ROLE_KEY
 - c. EDGE_ADMIN_SECRET (must match the value used by the iOS app)
 - d. PUSHY_SECRET_KEY (required for /notify)
5. Open the Edge Function code editor/console.
6. Copy the full contents of Middleware/supabase-edge-function.js into the code editor.
7. Click **Deploy**.

Pushy Account Setup (for iOS Push Notifications)

Pushy Dashboard

1. Create a Pushy account and create/select the Pushy app for this project.
2. Pushy Dashboard > your app > **API Authentication** > copy the **Secret API Key**.

APNs (iOS)

1. Apple Developer > Certificates, Identifiers & Profiles > Keys > create an **APNs Auth Key** and download the .p8.
2. In Pushy Dashboard, configure iOS/APNs using:
 - a. .p8 file
 - b. Key ID
 - c. Team ID
 - d. iOS bundle identifier (must match Xcode)

Backend configuration

1. Store the Pushy Secret API Key as a backend secret:
 - a. PUSHY_SECRET_KEY
2. Confirm the Supabase Edge Function environment has:
 - a. PUSHY_SECRET_KEY
 - b. SUPABASE_URL
 - c. SUPABASE_SERVICE_ROLE_KEY
 - d. EDGE_ADMIN_SECRET

Verify

1. Run the iOS app on a physical device.
2. Sign up / log in and confirm the device token is stored in public.pns.
3. Trigger /notify and confirm the notification is delivered.

Note: Never expose PUSHY_SECRET_KEY in the iOS app.